

Bom dia (ou boa tarde) a todos. Meu nome é George Gomes da Silva e, nos próximos minutos, vou apresentar o trabalho intitulado “Explicando Decisões de LLMs via Otimização Inversa”.

A pergunta central que move essa dissertação é aparentemente simples, mas tem implicações profundas: quando um Modelo de Linguagem de Grande Porte toma uma decisão de classificação, existe por trás dessa decisão um critério estável, algo que possamos chamar de uma “função objetivo implícita”?

Agradecimento inicial à banca e aos presentes pela oportunidade de submeter esse trabalho à avaliação. O plano de hoje é percorrer motivação, metodologia, os três algoritmos de otimização inversa, os resultados experimentais e as conclusões, mantendo sempre o rigor estatístico exigido.

LLMs estão se tornando ferramentas cotidianas em domínios de altíssima responsabilidade: triagem médica, análise de crédito, seleção de candidatos, apoio jurídico. Mas quando perguntamos “por que o modelo decidiu dessa forma?”, a resposta em linguagem natural não é, por si só, uma explicação verificável — o modelo pode estar racionalizando a resposta, sem refletir o processo real.

O que proponho aqui é inverter a pergunta. Em vez de pedir ao modelo que explique suas decisões, eu **observo as decisões** e, a partir delas, tento **recuperar matematicamente** o critério que as gerou. Essa é a essência da otimização inversa, proposta formalmente por Ahuja e Orlin em 2001 para problemas de decisão em geral.

Se o LLM **tem** um critério estável, essa métrica recuperada deve ser **transferível**: ao aplicá-la em novos problemas, ela precisa prever o que o LLM fará. É exatamente isso que testo experimentalmente nesta dissertação.

Cinco hipóteses estruturam o trabalho. A primeira, H1, afirma que o LLM mantém um critério estável — a métrica estimada a partir do Problema A prevê as classificações nos Problemas B e C com Kappa superior a 0,7. A segunda, H2, diz que fornecer exemplos rotulados amplifica a concordância entre LLM e métrica.

A terceira hipótese, H3, inverte os papéis: o LLM deve conseguir aprender o critério de um perito externo via few-shot. A quarta, H4, é a mais provocadora: exemplos difíceis deveriam ensinar melhor do que fáceis — vamos ver que os dados refutam essa hipótese de forma clara e surpreendente.

A quinta, H5, é de ordem estatística: o comportamento deve ser reproduzível entre múltiplas sementes e repetições. Para isso, rodo três sementes distintas com três repetições cada, além de bootstrap, Wilcoxon e tamanho de efeito de Cohen.

Trato cada classificação do LLM como a saída de um classificador centróide ponderado por uma métrica de Mahalanobis **diagonal**. Ou seja, para cada ponto o LLM estaria escolhendo o cluster mais próximo segundo uma distância ponderada — e eu quero recuperar os pesos dessa distância.

A escolha de métrica diagonal não é arbitrária. Em problemas 2D ela tem apenas dois parâmetros, w_1 e w_2 , o que garante identificabilidade, visualização intuitiva e comparação limpa entre algoritmos. É um mundo em que “sei o que deveria estar acontecendo”, o que permite testar os métodos com confiança antes de pensar em generalizar.

Três algoritmos independentes estimam essa métrica: Perceptron Estruturado com Relaxação de Margem, Mínimos Quadrados Não-Negativos via NNLS, e Programação Linear Max-Margin. Se os três convergem para a mesma solução, o achado é robusto ao método de estimação.

Trabalho em $\mathbb{R}^2$ propositalmente. Em duas dimensões consigo visualizar tudo — as classes, a fronteira de decisão, os erros e o próprio vetor w aprendido como inclinação geométrica.

O Problema A é a “bancada de treinamento”: 150 pontos distribuídos em duas classes gaussianas com centróides anisotrópicos, onde a direção x_2 é mais discriminativa do que a x_1 . Os Problemas B e C mantêm a mesma *ideia* de decisão mas com geometria alterada (deslocamento vertical, distorção), testando se o critério sobrevive. O Problema D muda completamente o jogo: um perito externo, com pesos que **eu** conheço, rotula os dados, e agora o LLM precisa aprender.

Tudo isso em três sementes aleatórias distintas, com CSVs exportados por seed — reprodutibilidade bit-a-bit independente do código.

No Problema A está o núcleo da Fase A do experimento. Geramos 150 pontos a partir de duas gaussianas com centróides em $(-1,5; 1,0)$ e $(1,5; -1,0)$, e pedimos ao LLM que classifique cada ponto em “A” ou “B”, *sem exemplos* (zero-shot). A temperatura da API é fixada em zero para minimizar estocasticidade — o que não é o mesmo que determinismo, mas reduz bastante a variância.

Sobre as 150 decisões coletadas, rodo simultaneamente os três algoritmos de otimização inversa. A saída é, para cada algoritmo, um vetor $w = (w_1, w_2)$ que, aplicado como métrica de Mahalanobis diagonal, *maximiza a concordância* com as decisões do LLM — a chamada **fidelidade** no próprio Problema A.

Esse w é a “assinatura” do critério do LLM no Problema A. O resto do experimento pergunta: até onde essa assinatura viaja?

O Problema B é o primeiro teste verdadeiro da hipótese de consistência. Mantenho a estrutura do problema — duas classes gaussianas anisotrópicas — mas deslocamento os centróides verticalmente (por exemplo, $(-1,5; 2,0)$ e $(1,5; 0,0)$). A decisão geométrica é diferente, mas a *ideia* é a mesma: preferência por x_2 .

Aqui eu não treino métrica nova. Pego o w aprendido no Problema A e o aplico diretamente em B, medindo quanto ele concorda com o LLM. Se o LLM é consistente, essa concordância deve ser alta — a métrica *sabe* o que o LLM vai fazer, mesmo sem ter visto os pontos.

Esse é o teste preditivo central: não basta o ajuste ser bom no conjunto de treino; ele precisa *generalizar* para uma nova geometria.

O Problema C empurra mais longe. A distorção geométrica é mais severa: os centróides estão em posições que não seguem a mesma simetria do Problema A, e a fronteira ideal não é simplesmente uma rotação da fronteira original.

A hipótese é que, mesmo assim, um critério *verdadeiro* deve produzir previsões consistentes. Se o w do Problema A continua fiel ao LLM no Problema C, estou observando algo que vai além de um fit local — é uma regra.

Os resultados aqui são chave: veremos mais adiante que a consistência cai modestamente de B para C (78,2% para 75,6%), o que já é um sinal positivo forte para H1, mas com o cuidado de reconhecer o que os números realmente dizem.

O Problema D inverte os papéis. Em vez de eu tentar explicar o LLM, eu pergunto: o LLM consegue aprender o critério de outra pessoa?

Construo um “perito externo”: um classificador centróide com uma métrica W_{expert} conhecida. Esse perito rotula 150 pontos. Agora eu dou ao LLM alguns desses pontos já rotulados — cinco, dez, vinte ou quarenta exemplos — e peço que ele classifique pontos novos reproduzindo as decisões do perito.

É aqui que testo a hipótese H3 (“LLM como aprendiz”) e também a hipótese H4 (“exemplos difíceis ensinam mais”), que será refutada empiricamente. Essa fase é crucial porque responde uma pergunta prática: se eu tiver poucos exemplos rotulados, o LLM é um bom aprendiz via few-shot?

Antes de mostrar resultados, vale explicitar as decisões metodológicas. O modelo escolhido é GPT-4o-mini, pela relação custo-desempenho e pela estabilidade de API. Temperatura zero para reduzir estocasticidade.

Uso três sementes aleatórias (42, 123 e 7), com três repetições por configuração, cobrindo os tamanhos de few-shot {0, 5, 10, 20, 40}. A coleta de decisões é **assíncrona**, com semaphore limitando a 10 chamadas paralelas — isso reduziu o tempo total de coleção em cerca de 10×.

Para garantir rigor, toda resposta do LLM passa por um parser de **sete camadas**, e respostas que ainda assim falham caem em um fallback determinístico por hash MD5 das coordenadas. Isso evita o viés geométrico que um fallback modular aritmético introduziria. No total, cerca de 114.478 chamadas ao LLM foram registradas nessa execução.

O primeiro algoritmo é um Perceptron Estruturado com Relaxação de Margem. A ideia é atualizar iterativamente os pesos w sempre que uma restrição de margem γ é violada, com busca binária no próprio γ para encontrar o maior valor que ainda deixa o problema viável.

É o método que mais lembra o Perceptron clássico, mas com a adaptação para o caso estruturado de otimização inversa. Tem hiperparâmetros — η , C e $\Delta\gamma$ — o que é bom para flexibilidade mas ruim para reprodutibilidade às cegas.

É também o algoritmo inspirado diretamente no trabalho de Medvet et al. sobre aprendizado de função objetivo, o que o torna o link mais direto com a literatura de otimização inversa aplicada a decisões humanas.

O segundo algoritmo é uma formulação como Mínimos Quadrados Não-Negativos. A intuição é direta: para cada exemplo, escrevo uma equação que diz “a distância ao centróide correto deveria ser menor que a distância ao errado”, e transformo isso em um sistema linear sobredeterminado. Os pesos w são obtidos minimizando o resíduo dessa aproximação, com a restrição de positividade.

O charme do NNLS é ser **zero-hiperparâmetro**. Uso `scipy.optimize.nnls`, que é uma implementação clássica e estável. Isso significa que o resultado é totalmente determinístico uma vez fixados os dados.

A limitação é que o objetivo é de mínimos quadrados, não de margem — mas em regime de baixa dimensão e grande número de exemplos, ele tende a concordar com os métodos de margem.

O terceiro algoritmo é o mais próximo da formulação canônica de Ahuja e Orlin: um programa linear que maximiza a margem γ sujeito às restrições de que, para cada exemplo, a combinação linear ponderada das diferenças quadráticas em relação aos centróides classifique corretamente, com margem de pelo menos γ .

Adiciono uma restrição de normalização ($\sum w = 1$) para evitar soluções degeneradas, e resolvo via `scipy.linprog`. A grande vantagem: **ótimo global garantido** e **zero hiperparâmetros**. Essa é a formulação que considero a mais canônica das três.

Rodar os três em paralelo me dá três w 's independentes. A similaridade de cosseno entre eles é um teste de *sanidade* do método: se concordam, o achado é robusto; se divergem, há algo estranho na fonte de dados.

Um ponto técnico que costuma escapar em trabalhos com LLM é a robustez da coleta. O que fazer quando o modelo retorna uma resposta malformada? Se você simplesmente descartar, está introduzindo um viés não-aleatório.

Por isso implementei um parser de sete camadas: busca por rótulos exatos, busca case-insensitive, busca por variantes (A, a, Classe A, etc.), por JSON, por prefixo, por sufixo, e por inclusão em texto livre. Cada camada é tentada em ordem. Em cinco retries de reformatação do prompt, quase todas as respostas conseguem ser parseadas.

O que sobrar após isso cai em um fallback determinístico por **MD5 das coordenadas**. A escolha do MD5 é deliberada: ele é uniforme e não correlacionado com a geometria do problema, ao contrário de um fallback aritmético modular que poderia introduzir viés espacial.

A Fase A é zero-shot pura. O LLM recebe um prompt descrevendo o problema e as coordenadas de um ponto, sem nenhum exemplo. Responde “A” ou “B”. Isso isola o critério *pré-existente* do modelo — o que ele traz de fábrica.

Os três algoritmos são rodados sobre as mesmas 150 decisões, produzindo w_{perc} , w_{nnls} e w_{lp} . A fidelidade (quantas classificações a métrica reproduz) é medida *no próprio Problema A*. Também calculo a similaridade de cosseno entre os três w 's para verificar concordância dos métodos.

Este é o “passo zero” do experimento: se aqui já houvesse discordância grande entre algoritmos, tudo o que vem depois estaria comprometido. Por isso, dedico slides específicos à análise interna da Fase A.

Os números principais. O w médio aprendido pelo Perceptron é da ordem de $(0,29; 0,71)$, indicando claramente que o LLM dá peso maior à coordenada x_2 — consistente com a anisotropia dos dados.

As fidelidades no Problema A são altas e muito próximas entre os três algoritmos. Isso é exatamente o que esperávamos ver como sanidade. A similaridade de cosseno entre Perceptron e NNLS atinge 0,986, e entre os três pares ela fica acima de 0,96 na maior parte das configurações.

A leitura é: independente de qual algoritmo uso, o critério recuperado do LLM é essencialmente o mesmo vetor. Esse é o primeiro apoio empírico à ideia de que o LLM tem um critério estável e identificável a partir de suas decisões.

Quando a métrica aprendida discorda do LLM, onde isso acontece? Esse slide mostra a análise quantitativa: dividimos o plano em regiões de alta margem e baixa margem (perto da fronteira), e contamos erros.

Para a semente 42, cerca de 74,1% dos erros da métrica estão concentrados na região de baixa margem — ou seja, pertinho da fronteira, onde até um humano titubearia. Isso é muito informativo: não são erros aleatórios espalhados pelo domínio; são disputas justamente onde a fronteira é ambígua.

Essa concentração de erros na zona de ambiguidade é evidência de que a métrica está capturando o critério “macroescópico” do LLM, com diferenças só nas áreas de baixa confiança — exatamente o tipo de robustez que um bom estimador deve ter.

Aqui olho a *variabilidade* de w . Se o critério é estável, w deve ter pouca variância entre sementes.

A similaridade de cosseno *entre sementes* fica em média 0,984, com range estreito [0,975; 0,992]. Isso é praticamente colinearidade.

Olhando pela razão w_1/w_2 , a média é 0,405 com intervalo bootstrap 95% [0,318; 0,458]. O coeficiente de variação em zero-shot é de 32% e cai para apenas 6% quando oferecemos few-shot ≥ 5 . Traduzindo: o LLM já tem um critério latente, e poucos exemplos bastam para “focar” esse critério de maneira ainda mais ajustada. Esse é o primeiro sinal forte em favor da H2 (few-shot amplifica).

Este gráfico mostra tudo junto: fidelidades, transferências para B e C, e um scatter plot dos w 's por algoritmo. O scatter é a parte mais importante — os pontos praticamente se sobrepõem, confirmando visualmente a convergência entre Perceptron, NNLS e LP.

É importante notar que os três algoritmos foram escolhidos *exatamente* por serem diferentes. Perceptron é iterativo e depende de hiperparâmetros; NNLS é fechado e sem parâmetros; LP tem ótimo global. Quando três métodos tão distintos chegam ao mesmo vetor, a dedução lógica é que o sinal *está nos dados*, não no método.

Esse é um dos pilares da credibilidade dos achados: não é um artefato de otimização.

O Perceptron tem três hiperparâmetros. Para provar que a escolha específica não está determinando o resultado, varri uma grade de valores para η , C e $\Delta\gamma$.

Em todas as configurações testadas, a similaridade de cosseno entre o w resultante e o w padrão se manteve acima de 0,86, com a maioria dos casos acima de 0,95. A fidelidade no Problema A variou em menos de dois pontos percentuais.

Isso reforça: o critério recuperado **não depende** do ajuste fino dos hiperparâmetros. É uma propriedade estrutural do que o LLM está fazendo, não uma idiosincrasia do otimizador.

Passamos à parte preditiva. A métrica aprendida na Fase A é *transportada* e aplicada ao LLM classificando o Problema B e depois o Problema C. A métrica agora funciona como um modelo substituto do LLM — é o teste crucial de H1.

A instância do teste é simples: o LLM classifica os pontos novos, a métrica também classifica os mesmos pontos, e comparamos com consistência, Kappa de Cohen e F1. Kappa é particularmente adequado porque corrige pela concordância esperada ao acaso em problemas binários.

O interessante é que *não uso* nenhuma informação do Problema B ou C para treinar. O w vem todo do Problema A. Qualquer concordância alta aqui é uma vitória da hipótese de consistência.

Números agregados sobre todas as sementes e repetições: a consistência média no Problema B é de 78,2% com desvio de 12,1%. No Problema C cai para 75,6% com desvio de 14,5%. Kappa médio passa de 0,56 (B) para 0,51 (C).

Esses são números *significativos*. Para comparação, um preditor que respondesse ao acaso teria consistência de 50% e Kappa zero. Estamos bem acima do acaso, com desvio que reflete a variabilidade natural entre sementes mas não corrompe a significância.

Importante destacar: 78% não é 100%. A métrica *não* é o LLM. Ela é uma aproximação do critério — uma maquete, não o prédio. Veremos mais à frente por que isso é esperado e como isso *ainda assim* sustenta H1.

A tabela detalhada por n -shot revela tendências finas. Em zero-shot puro a consistência em B é de cerca de 73%; sobe para 84% com cinco exemplos; oscila entre 75% e 84% conforme aumenta o número de exemplos.

Há um platô: acima de 5 exemplos o retorno marginal é baixo. Isso é prático: se você conseguir rotular cinco bons pontos, o LLM já extrai a maior parte do sinal disponível.

O F1 acompanha o padrão da consistência. Kappa mostra variação maior entre sementes, mas sempre em regime positivo — nunca tivemos Kappa negativo, o que indicaria concordância pior que acaso.

Como interpretar esses números? Eu gosto de usar a analogia do “shadow model”. A métrica é uma sombra que segue o LLM ao longo dos problemas. Se a sombra andar mais ou menos junto com o LLM, é porque há algo real produzindo tanto a sombra quanto o LLM — o critério decisional.

Não estou dizendo que a métrica é o LLM. Estou dizendo que ela captura uma fração muito significativa do seu comportamento, e essa fração é transferível. O gap de 20% a 25% é exatamente a diferença entre um “modelo centróide ponderado” e o “LLM real”.

A hipótese H1 é sustentada empiricamente por todos os três problemas, três sementes e três repetições, com a devida humildade sobre os limites do método.

Um dos achados colaterais mais úteis desse trabalho veio de uma pergunta simples: e se eu listar “B ou A” em vez de “A ou B” no prompt? A intuição sugeriria que não faz diferença — o LLM conhece as classes.

Não é o que acontece. A mudança na **ordem** das classes no prompt altera o desempenho em até 25 pontos percentuais, dependendo da configuração. O LLM tem um *viés de posição* real, prefere a primeira opção citada em contextos de incerteza.

Esse achado não é o foco principal da dissertação, mas é um alerta metodológico importante: qualquer estudo com LLMs em classificação precisa controlar ordem de apresentação das classes, sob pena de medir artefato.

O gráfico mostra lado a lado as barras das duas ordens, para os quatro pares de nomes testados (A/B, 0/1, Positivo/Negativo, Azul/Vermelho). Em todos os casos existe um gap — alguns menores, outros francamente grandes.

O caso mais dramático é justamente o A/B numérico, onde a inversão para B/A muda a classificação de perto de um quarto dos pontos. Já com rótulos semânticos carregados (“Positivo/Negativo”), o efeito é menor mas ainda presente.

A recomendação prática é usar ordem aleatorizada nos prompts ou pelo menos reportar sensibilidade. Esse slide é um dos mais úteis para a comunidade aplicada que usa LLMs em classificação.

Análogo ao slide anterior, mas para os nomes das coordenadas. Se em vez de “ x_1, x_2 ” eu usar “altura, peso” ou “feature_1, feature_2”, o LLM muda seu comportamento?

Resposta: sim, modestamente. Os w 's aprendidos são similares entre versões com nomes diferentes (cosseno acima de 0,92), mas as fidelidades oscilam em 2 a 5 pontos percentuais. Com nomes semânticos carregados, o LLM parece trazer priors do domínio — por exemplo, “peso” tende a ser associado a uma direção específica.

O achado é consistente com a literatura recente sobre *priming* em LLMs: o vocabulário importa. Para o objetivo desta dissertação, o tranquilizador é que o *sinhal central* (direção do w) sobrevive.

Quatro variações do prompt foram testadas: o padrão da dissertação, uma variante *geométrica* (que descreve o problema em linguagem espacial explícita), uma variante de *chain-of-thought* ("pense passo a passo"), e uma variante *tabular* (coordenadas apresentadas como tabela markdown).

Cada uma rodou a Fase A completa em todas as sementes. A pergunta: o critério recuperado muda conforme o prompt? Se mudar muito, o que estamos medindo é o prompt, não o modelo.

É um teste de *robustez de medição*. Mais crítico do que parece, porque na prática muitos trabalhos com LLMs usam um único prompt e reportam resultados como se fossem gerais.

Os w 's recuperados são **extraordinariamente consistentes** entre variantes. No scatter plot principal, as quatro nuvens de pontos se sobrepõem quase perfeitamente; a similaridade de cosseno média entre variantes fica acima de 0,94.

Fidelidades no Problema A variam em menos de 3 pontos percentuais entre variantes. A variante CoT dá um pequeno ganho em alguns casos, mas não de forma estatisticamente significativa dada a amostra.

Conclusão: o critério recuperado é **resistente ao phrasing**. Não estamos medindo um efeito de prompt específico; estamos medindo algo sobre o *critério* do LLM que sobrevive à reformulação. Esse é outro pilar da credibilidade do método.

Agora o LLM aprende. Temos um perito externo com $W_{\text{expert}} = [0,3; 1,5]$ (x2 dominante), que rotula os dados. O LLM recebe k desses exemplos no prompt e tem que reproduzir o padrão.

Medimos acurácia, Kappa e F1 em relação ao rótulo do perito. Escolhemos os exemplos segundo quatro estratégias distintas — que discutiremos adiante — e variamos k entre 0 e 40.

A pergunta guia é H3: o LLM é um bom aprendiz de critério via poucos exemplos? Se sim, a curva de aprendizado deve subir rapidamente e platou acima de 90% dentro de poucas dúzias de exemplos.

Quatro estratégias. “Easy” escolhe pontos longe da fronteira, com alta margem — pontos protótipos de cada classe. “Hard” escolhe próximos da fronteira, ambíguos. “Mixed” faz 50/50. “Random” é um baseline sem critério.

Minha hipótese inicial era H4: pontos difíceis devem ensinar mais, porque carregam informação sobre *onde está* a fronteira. É uma intuição antiga em aprendizado ativo.

Vamos ver que os dados falam outra coisa. E essa é uma das partes mais interessantes desta dissertação: uma hipótese nasce, é testada honestamente, e os dados a **refutam** de maneira clara.

Começando em 60,9% com zero-shot, a acurácia sobe rapidamente com few-shot. Com 5 exemplos já estamos em 83%; com 10 em 90%; com 20 em 94%; e com 40 em 97,7%.

Essa é uma curva muito estável. Kappa acompanha quase linearmente, e F1 permanece acima de 0,90 a partir de 10 exemplos.

Esse resultado, em si, sustenta H3 de forma forte: o LLM aprende efetivamente o critério do perito com muito poucos exemplos. Mas dentro dessa curva agregada mora a refutação de H4 — que aparece quando separamos as estratégias.

Agora separando por estratégia, a ordem dos painel é surpreendente. Random vem em primeiro lugar com média 90,3% ao longo dos n -shots; mixed em 87,8%; easy em 81,5%; e hard **em último, em 75,6%**.

Em todos os tamanhos de few-shot, a ordem relativa se mantém. A diferença easy-hard é grande: quase seis pontos percentuais médios, com significance estatística via Mann-Whitney U.

Esse slide é o ponto-chave da refutação de H4. Discutirei no próximo slide *por que* isso acontece, que é talvez a parte mais valiosa do trabalho de interpretação.

A explicação que proponho é a seguinte: em aprendizado ativo clássico, “hard” é informativo porque você está treinando um modelo paramétrico que converge para a fronteira. Pontos na fronteira refinam essa fronteira.

Mas o LLM em few-shot não faz isso. Ele não “treina”; ele **usa os exemplos como âncoras prototípicas**. Pontos difíceis ambíguos não fornecem âncora clara — ele pode achar que um ponto na fronteira é de uma classe, e confundir-se com outro igualmente próximo.

Pontos “easy” são exemplares canonicos: “assim é uma classe A, assim é uma classe B”. O LLM os usa como referência para classificar novos pontos por similaridade. É uma interpretação consistente com a literatura recente sobre in-context learning. **H4 refutada; random supera tudo** porque traz diversidade natural.

Para testar se o aprendizado não depende de um perito específico, repeti a Fase D com três configurações de expert: `aniso_x2` (original), `aniso_x1` (invertido, w_1 dominante), e `euclidean` ($w = [1, 1]$).

O LLM aprende os três com desempenho similar. Acurácias médias ficam entre 91% e 96% com 40 exemplos, independente de qual é a estrutura geométrica do perito.

É evidência forte para H3: o LLM não tem “preferência” por uma anisotropia específica; ele aprende o que você quiser ensinar. Isso sustenta o uso do LLM como classificador flexível em aplicações reais com peritos humanos atrás dos exemplos.

Uma questão levantada pelos orientadores: se pontos fáceis ensinam bem, e difíceis ensinam mal, o que acontece quando você *mistura* os dois?

O experimento de diluição fixa 4 exemplos *hard* e adiciona progressivamente exemplos *easy*: 0, 2, 4, 10, 16, 20. A acurácia *sobe monotonicamente*: os fáceis “resgatam” o desempenho que os difíceis estavam prejudicando.

O ponto interessante é que não há “limite de diluição” onde o *easy* começa a prejudicar. Cada *easy* adicional contribui um pouquinho, até que o efeito dos *hard* está efetivamente anulado. Isso reforça a interpretação prototípica do in-context learning.

Outro viés conhecido da literatura é o *recency bias*: LLMs pesariam mais o que vem no final do prompt. Testei isso mantendo os mesmos exemplos e mudando apenas a **ordem**: classe 0 primeiro, classe 1 primeiro, embaralhado, alternado.

O resultado surpreende: **não há efeito estatisticamente significativo**. O teste de Kruskal-Wallis para n -shot = 5, 10, 20, 40 retorna valores $p > 0,19$ em todos os casos.

Pelo menos para o GPT-4o-mini nesse tipo de problema, a ordem dos exemplos não importa para the desempenho agregado. Esse é um resultado *nulo* e honesto, e um resultado nulo também é um achado científico — não se confirmam todos os viés apregoados na literatura.

Validação fundamental: se os algoritmos de otimização inversa funcionam mesmo, eles precisam recuperar um W que *eu sei qual é*.

Gero dados sintéticos rotulados por um W oracle conhecido (por exemplo $[0,1; 2,0]$) e rodo os três algoritmos. Os cossenos recuperados em relação ao W verdadeiro são impressionantes: para semente 42 e anisotropia `aniso_x2`, o Perceptron chega a cosseno 0,9999, NNLS e LP também acima de 0,99.

Resultados semelhantes em todas as configurações anisotrópicas testadas. Isso é o “pisso de sanidade”: se falhasse aqui, não faria sentido confiar nos achados em dados de LLM. Não falha.

Teste adicional: aprendo W em uma configuração anisotrópica e aplico o mesmo W em outra configuração. Mede a queda quando a geometria muda, com W oracle conhecido.

Como esperado, há queda proporcional ao ângulo entre as anisotropias. Configurações compatíveis (aniso_x2 treino $\rightarrow$ aniso_x2 teste) mantêm consistência acima de 95%; configurações ortogonais (aniso_x2 $\rightarrow$ aniso_x1) caem para valores próximos do acaso.

Esse é um referencial quantitativo para interpretar os resultados das Fases B e C: os 78% e 75% do LLM são compatíveis com uma configuração cuja geometria tem certa rotação em relação ao treino — mas não total.

E se o LLM estivesse capturando uma interação entre x_1 e x_2 que uma métrica diagonal não consegue representar? Para testar isso, crio uma terceira feature $x_3 = x_1 \cdot x_2$ — essencialmente um kernel quadrático, sem sair do mundo linear.

Rodo três versões: LLM classificando com 2 features originais, LLM classificando com 3 features (a terceira explícita no prompt), e métrica aprendida no espaço 3D. Se o LLM já estava capturando interação implicitamente, não deve haver grande salto. Se não, o 3D ajuda.

É um teste de *expressividade*: o critério do LLM é genuinamente linear na métrica diagonal, ou há algo não-linear escondido?

O LLM em 2 features acerta 84,4% das classificações geradas pelo oracle quadrático. A métrica aprendida com 3 features (via NNLS) acerta 86,2%. O LLM em 3 features classifica diretamente em patamar similar.

A diferença é pequena. Isso sugere que, em regime anisotrópico e pouco complexo, **uma métrica diagonal captura a maior parte do sinal**. Não existe uma interação não-linear latente que um olhar R3 revelaria.

É um resultado confortante para a escolha metodológica: a simplificação diagonal não está deixando muito na mesa. Em problemas mais complexos (3D real, Iris), esse teste precisa ser refeito.

Para calibrar o desempenho do LLM contra classificadores simples, treino nos mesmos exemplos da Fase D três baselines: k-NN (com k ajustável até 5), Regressão Logística linear regularizada, e SVM com kernel RBF.

Uso exatamente os mesmos conjuntos de treino e teste. A pergunta é brutal: o LLM faz algo que um algoritmo trivial de 1985 não faria?

É uma pergunta que precisa ser feita, porque a narrativa *hype* sobre LLMs tende a esquecer que, em tarefas estruturadas, classificadores simples costumam ser imbatíveis.

A Regressão Logística atinge 93,4% no Problema D com 40 exemplos. O SVM-RBF chega a 87,5%. k-NN com $k = 5$ em 89,5%, e com $k = 3$ em 84%. O LLM em 97,7%.

A vantagem do LLM é real mas **modesta** — de 4 a 14 pontos percentuais dependendo do baseline. Em regime de *muitos exemplos*, a Regressão Logística fica próxima do LLM.

A leitura honesta: o LLM é competitivo, mas não revolucionário, em classificação supervisionada 2D. A vantagem principal do LLM está em outro lugar: **a flexibilidade** de não precisar de retreino quando o domínio muda. É um argumento de engenharia, não de acurácia.

Como comentei no início, uso três sementes (42, 123, 7) com três repetições cada. Esse slide sumariza a concordância entre seeds das métricas principais.

Em fidelidades, o range entre seeds é de 3 a 5 pontos percentuais. Em Kappa, 0,04 a 0,08. A direção de w é praticamente a mesma (cosseno $\approx 0,98$). Refutações de hipóteses aparecem em *todas* as sementes — não é artefato de uma semente específica.

Isso é H5 (estabilidade) confirmada. Os achados não dependem da semente; eles dependem da estrutura do problema e do modelo.

Todas as afirmações comparativas são acompanhadas de testes estatísticos. Bootstrap com 10 mil reamostragens para intervalos de confiança em todas as fidelidades e consistências.

Wilcoxon signed-rank para comparações pareadas (por exemplo, easy vs. hard no mesmo par semente-repetição), que dá $p < 0,01$ para a refutação de H4. Tamanho de efeito de Cohen's d acima de 0,6 (efeito médio a grande) na mesma comparação.

Mann-Whitney U para comparações não-pareadas; correlação ponto-biserial para n -shot vs. acurácia; Kruskal-Wallis para comparações de quatro grupos simultâneos (como as ordens de exemplos).

Além dos principais, alguns testes complementares. A correlação ponto-biserial entre número de exemplos e acurácia na Fase D é de $\approx 0,72$ — forte. Mann-Whitney U para easy vs. hard: $p < 0,001$, U grande.

Kruskal-Wallis para as quatro estratégias de exemplos no $n\text{-shot} = 40$: $p < 0,01$, indicando que pelo menos uma estratégia difere das outras — e o post-hoc confirma que hard é a que puxa o grupo para baixo.

Todos os p-valores são reportados em tabelas nos CSVs. Cross-check entre métodos paramétricos e não-paramétricos sempre feito, já que as distribuições de acurácia não são gaussianas.

Cada uma das três sementes tem um dashboard próprio — figura 19 em cada seed. Mostram lado a lado: scatter do problema, fronteira aprendida, matriz de confusão, distribuição de erros por região e curva de aprendizado.

São essenciais para inspeção visual. Um erro estatístico típico é olhar só médias agregadas e perder heterogeneidade entre sementes. Nesses dashboards, dá para ver que, apesar de ligeiras diferenças numéricas, o *padrão visual* é consistente entre as três sementes.

São salvos como PNG separados, disponíveis no repositório da execução. Podem ser mostrados sob demanda durante a banca se houver perguntas específicas sobre alguma seed.

Recapitulando: H1 (consistência) é **sustentada** com Kappa médio 0,53 em problemas não vistos. H2 (few-shot amplifica) também **sustentada**: CV da razão w_1/w_2 cai de 32% para 6% com exemplos.

H3 (LLM como aprendiz) **sustentada** de forma forte: acurácia chega a 97,7% e o resultado se replica em três peritos distintos. H4 (hard > easy) **refutada**: 75,6% vs. 81,5%, com significância estatística.

H5 (estabilidade) **sustentada**: três sementes produzem resultados qualitativamente idênticos, com variação numérica pequena em métricas centrais. Quatro de cinco hipóteses confirmadas e uma refutação informativa — um balanço saudável para um trabalho empírico.